VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: File Allocation Strategies

Experiment No: ____ Date: _____

Aim: Simulate all file allocation strategies. A. Sequential, B. Indexed, C. Linked

A. Sequential

Algorithm:

1. **Start**.
2. Input disk block status, total number of blocks, starting block, and file length.
3. Check whether all blocks from start to start + length - 1 are free.
4. If all blocks are free, allocate them to the file.
5. Display the allocation status and allocated block numbers.
6. **Stop**.

B. Indexed

Algorithm:

1. **Start**.
2. Input index block number and number of file blocks n.
3. Check whether the index block is free.
4. Input the required block numbers and check whether all blocks are free.
5. If all blocks are free, allocate the index block and all required blocks.
6. Display the index block and its allocated block mapping.
7. **Stop**.

C. Linked

Algorithm:

1. **Start**.
2. Input starting block number and number of blocks n.
3. Input the block numbers forming the linked chain.
4. Check whether all required blocks are free.
5. Allocate the blocks and link them in sequence, ending with -1.
6. Display the linked block sequence.
7. **Stop**.

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

_____

Experiment No: ____ **Date:** _____

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

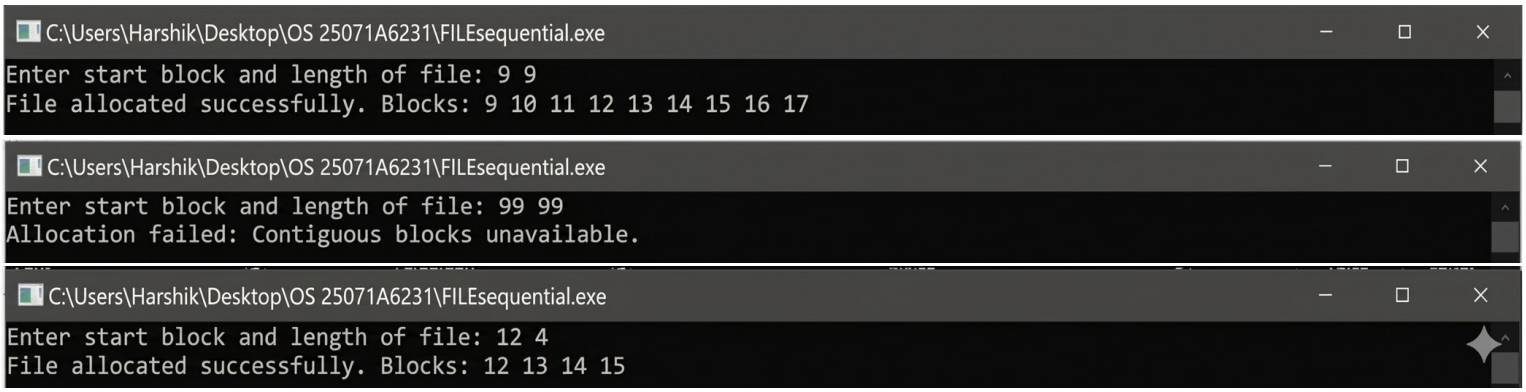
A. Sequential File allocation

Program:

```
#include <stdio.h>

int main() {
    int disk[50] = {0}, start, len, alloc = 1;
    printf("Enter start block and length of file: ");
    scanf("%d %d", &start, &len);
    for (int i = start; i < start + len; i++) if (disk[i]) alloc = 0;
    if (alloc) {
        printf("File allocated successfully. Blocks: ");
        for (int i = start; i < start + len; i++) {
            disk[i] = 1; printf("%d ", i);
        }
        printf("\n");
    } else printf("Allocation failed: Contiguous blocks unavailable.\n");
    return 0;
}
```

Testcases:



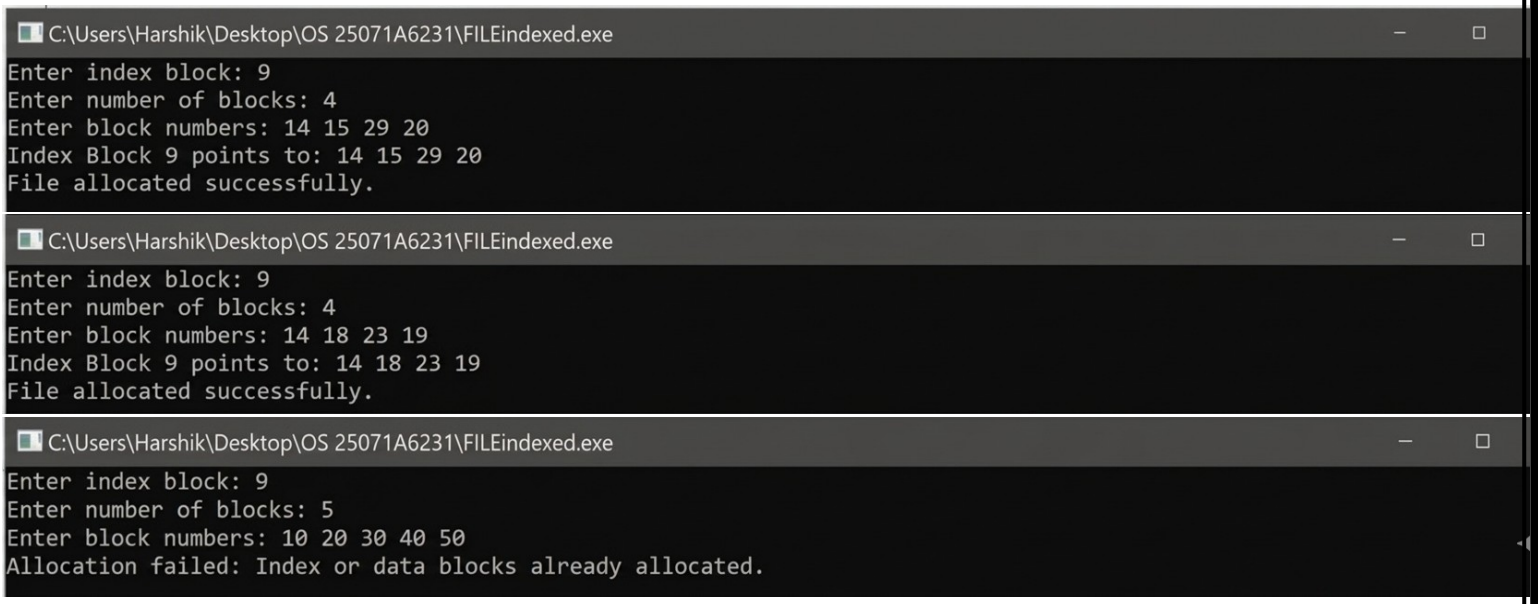
B. Indexed File Allocation

Program:

```
#include <stdio.h>

int main() {
   int disk[50] = {0}, idx, n, b[20], alloc = 1;
   printf("Enter index block: "); scanf("%d", &idx);
   printf("Enter number of blocks: “); scanf("%d", &n);
   printf(“Enter block numbers: ");
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   if (disk[idx]) alloc = 0;
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      disk[idx] = 1; printf("Index Block %d points to: ", idx);
      for (int i = 0; i < n; i++) { disk[b[i]] = 1; printf("%d ", b[i]); }
      printf("\nFile allocated successfully.\n");
   } else printf("Allocation failed: Index or data blocks already allocated.\n");
   return 0;
}
```

Testcases:



C. Linked

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6231\n");
   int disk[50] = {0}, n, b[20], alloc = 1;
   printf("Enter number of blocks:  "); scanf("%d", &n);
   printf("Enter block sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      printf("Linked Allocation Sequence: ");
      for (int i = 0; i < n; i++) {
         disk[b[i]] = 1; printf("%d -> ", b[i]);
      }
      printf("NULL\nFile allocated successfully.\n");
   } else printf("Allocation failed: One or more blocks already allocated.\n");
   return 0;
}
```

Testcases:

